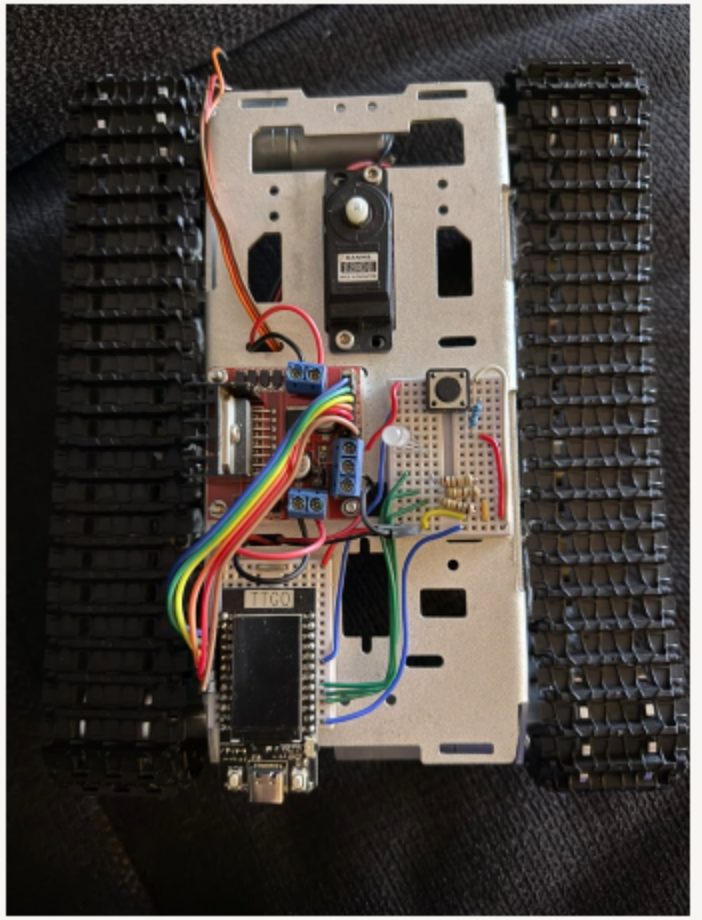
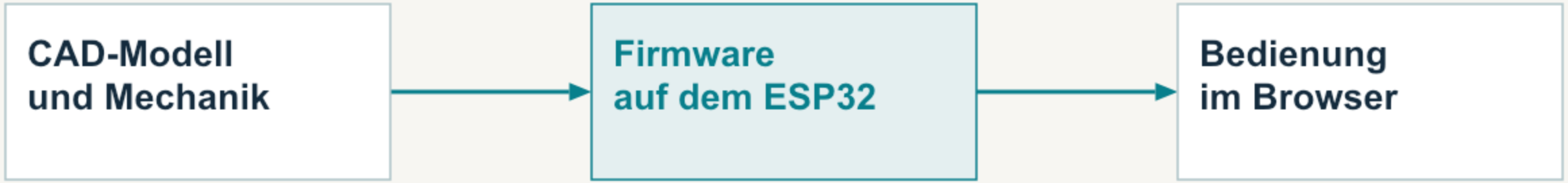
Ein Tank im Browser



ESP32, Webserver und verständlicher C++-Code

Eingebettete Systeme
All in with KI

Unser Projekt bekommt eine Oberfläche



Heute: Daten lesen, Befehle senden und Code verständlich aufteilen

Arduino und ESP32: die Begriffe

Arduino

Boardfamilien

IDE und Bibliotheken

Framework für setup() und loop()

ESP32

Mikrocontroller-Familie

Boards von verschiedenen Herstellern

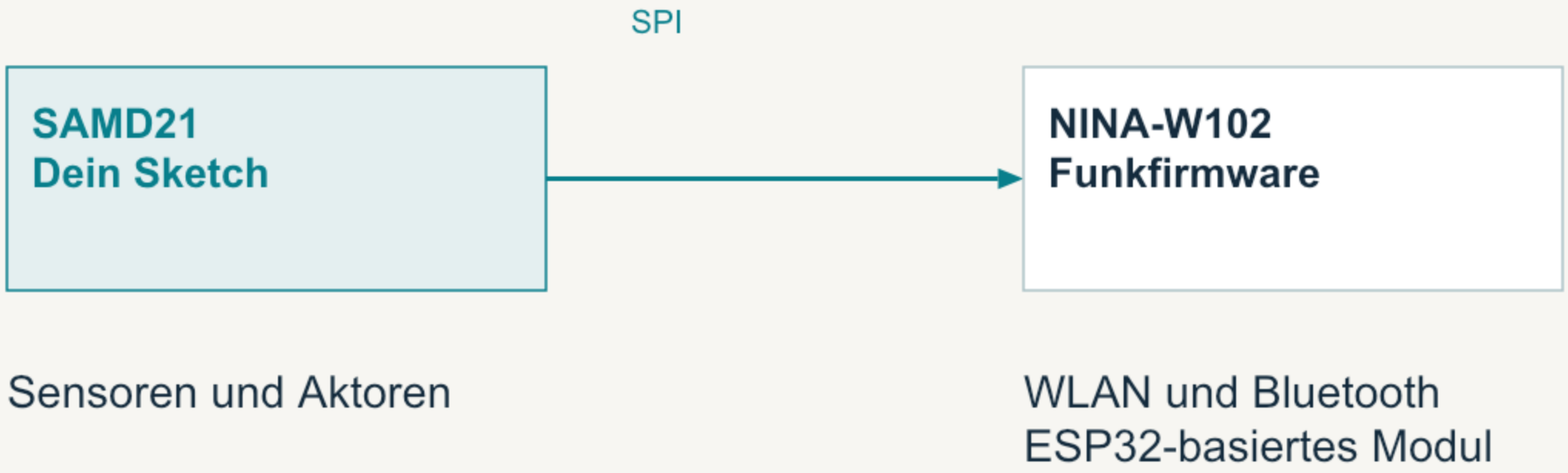
Unser Code nutzt das Arduino-Framework

Drei konkrete Board-Konzepte

	UNO R3	MKR WiFi 1010	Unser TTGO ESP32
Anwendung	ATmega328P	SAMD21	ESP32
WLAN	Zusatzhardware	NINA-W102-Modul	Im ESP32 integriert
GPIO-Logik	5 V	3,3 V	3,3 V
Prinzip	MCU ohne Funk	MCU + Funkmodul	Anwendung + Funk

Die Aussage „Arduino hat kein WLAN“ wäre zu pauschal.

MKR WiFi 1010: zwei Prozessorrollen



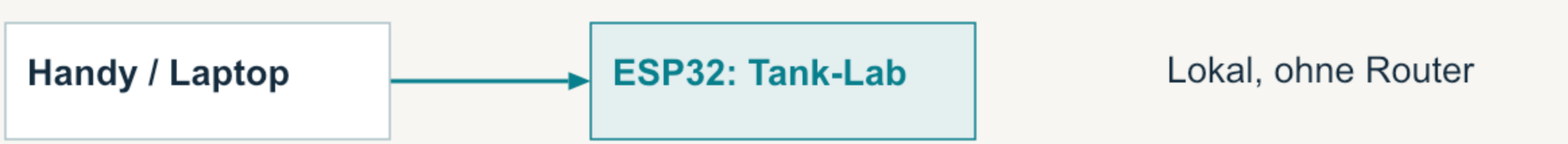
Bei unserem Tank laufen Anwendung und WLAN direkt auf dem ESP32.

Der ESP32 verbindet Hardware und Netzwerk



Zwei WLAN-Betriebsarten

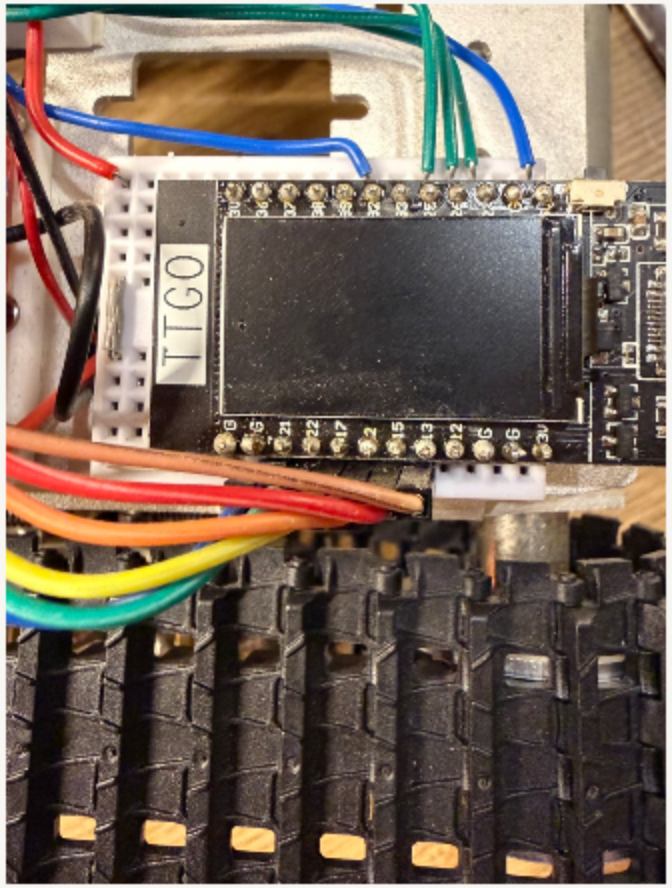
Access Point: unser Beispiel



Station: spätere Erweiterung



Der Tank als eingebettetes System



TTGO mit ESP32 und Farb-TFT

Taster als Eingang

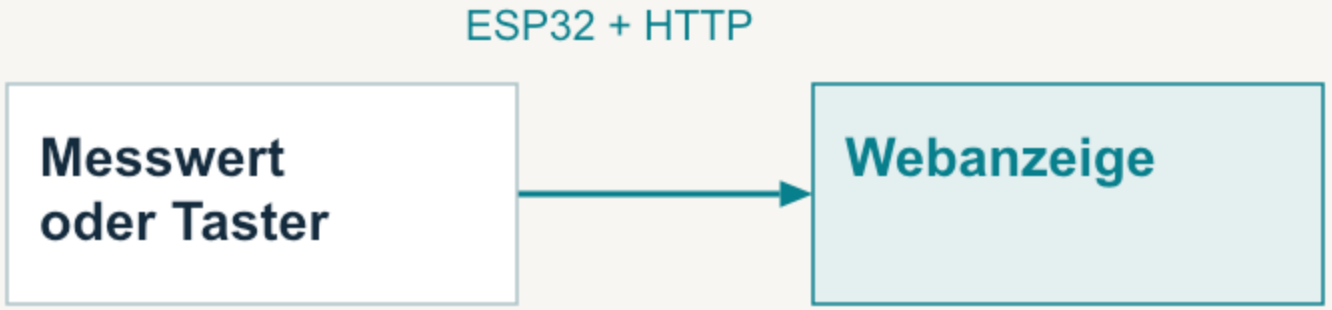
RGB-LED als Ausgang

L298N für die Motorleistung

Zwei Ketten, getrennt steuerbar

Auch ein Sensorprojekt nutzt dasselbe Prinzip





Wo liegt der Wert?
Wie kommt er zum Browser?

Wo läuft welcher Code?

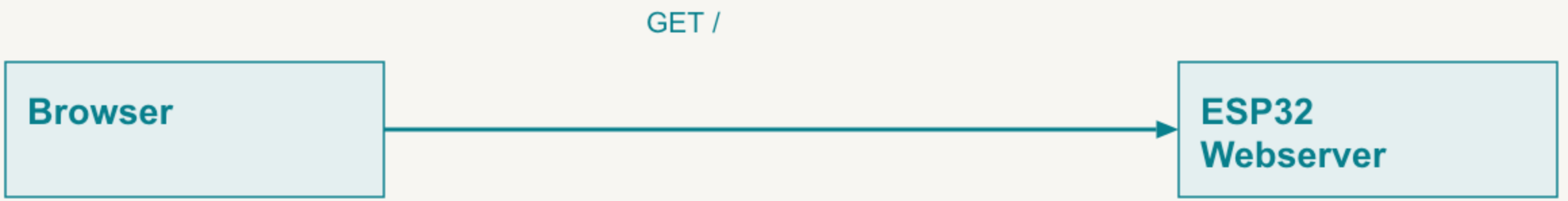


HTML: Inhalt
CSS: Gestaltung
JavaScript: Bedienung und fetch()

C++: Webserver und Routen
Klassen: Hardware ansteuern
Flash: Text der Webseite

Speicherort der Datei und Ort ihrer Ausführung sind verschiedene Dinge.

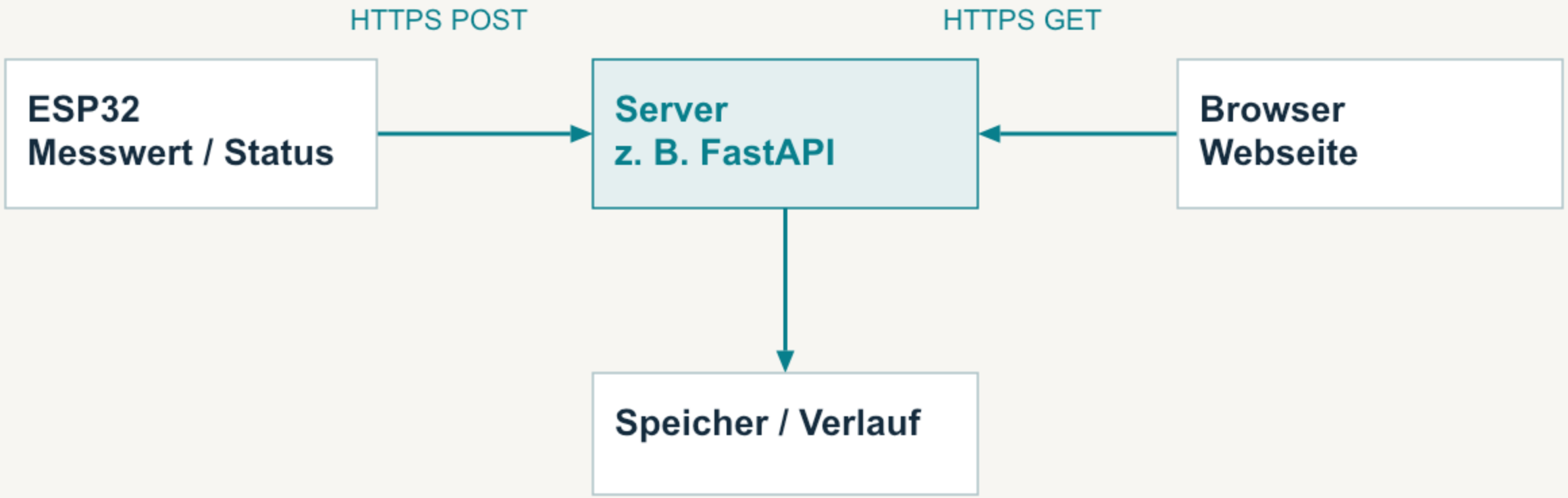
Die Webseite direkt auf dem ESP32



1. HTML, CSS und JavaScript laden
2. Status als JSON abrufen
3. Aktionen per POST senden

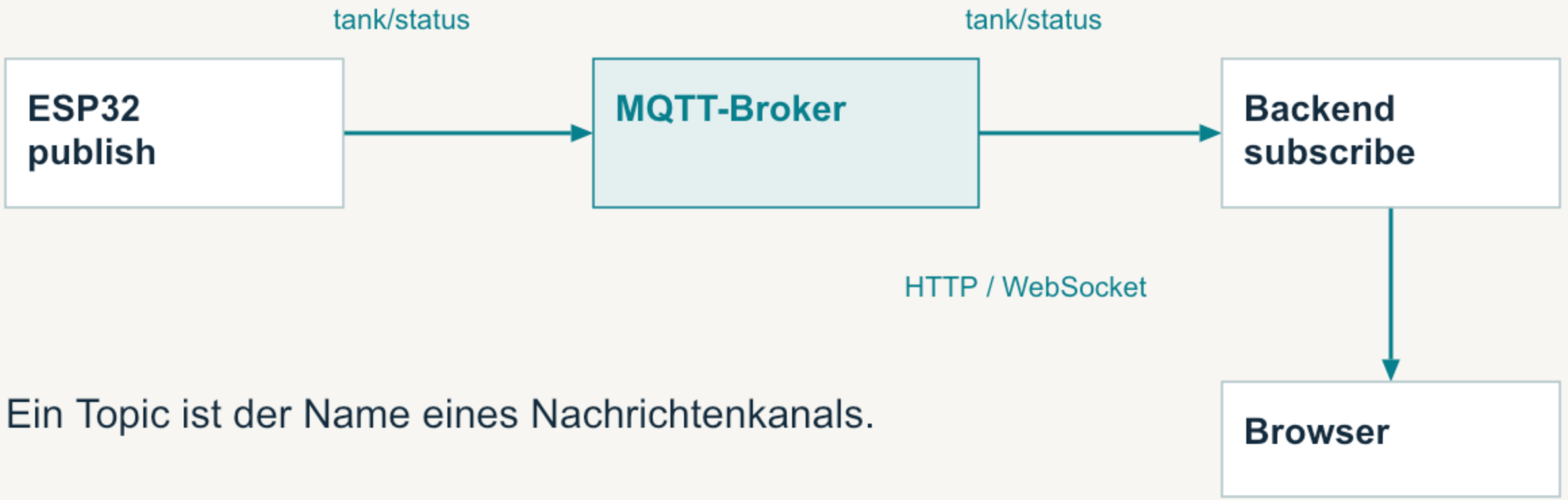
Adresse in unserem Geräte-WLAN: http://192.168.4.1

Webseite und Daten auf einem Internetserver



Ausblick: FastAPI läuft auf dem Server, die Hardwaresteuerung bleibt auf dem Gerät.

MQTT: Nachrichten über einen Broker



Ein Topic ist der Name eines Nachrichtenkanals.

Welche Architektur passt zum Projekt?

Variante	Typische Aufgabe	Zusätzlicher Baustein
ESP32-Webserver	Lokale Bedienung	Browser im erreichbaren Netz
Server mit HTTP-API	Verlauf, mehrere Nutzer	Server und Datenspeicher
MQTT mit Backend	Viele Geräte / Nachrichten	Broker und Webanbindung

Heute bauen wir die erste Variante. Die anderen sind mögliche Erweiterungen.

Live-Demo 1: HelloWebServer

Ein WLAN. Zwei Routen.

/ zeigt „Hallo vom Tank!“

/api/hello liefert JSON

Projekt: examples/01_HelloWebServer

PlatformIO-Umgebung: hello_webserver

Ein Webserver in wenigen Zeilen

```
WebServer server(80);

void setup() {
  WiFi.mode(WIFI_AP);
  WiFi.softAP(Config::WifiName,
              Config::WifiPassword);
  // Routen anmelden ...
  server.begin();
}
```

Port 80: HTTP

Eigenes WLAN

Routen vor begin()

Ausschnitt: Fehlerbehandlung und Hardwareinitialisierung stehen im vollständigen Beispiel.

Eine Route verbindet URL und Funktion

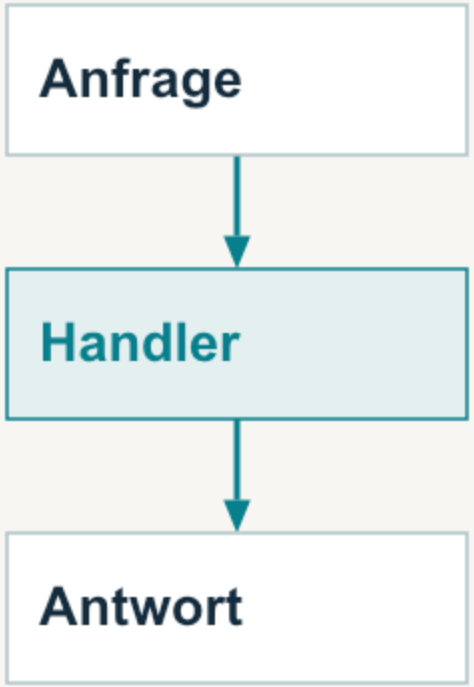
```
server.on("/api/hello", HTTP_GET, []() {
  server.send(200, "application/json",
    "{\"message\":\"Hallo\",\"device\":\"ESP32\"}");
});
```

URL + Methode bestimmen den Handler

Der Handler erzeugt eine HTTP-Antwort

Warum loop() weiterlaufen muss

```
void loop() {
  server.handleClient();
  delay(1);
}
```



Lange Wartezeiten im Hauptprogramm verzögern die Webbedienung.

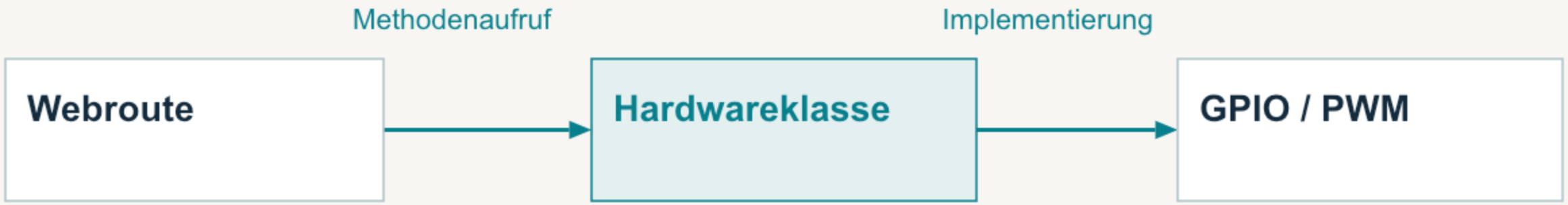
Live-Erweiterung: eine eigene Route

```
server.on("/api/team", HTTP_GET, [](){
  server.send(200, "application/json",
    "{\"team\":\"Team Embedded\"}");
});
```

Danach: http://192.168.4.1/api/team

Neue Route als Übung ergänzen, dann erneut kompilieren und hochladen.

Mehr Hardware braucht klare Zuständigkeiten



Webseite ändern, ohne den Motortreiber neu zu verdrahten

Pinbelegung an einer benannten Stelle nachschlagen

Unser Projekt ist nach Aufgaben aufgeteilt

Datei / Klasse	Verantwortung
Pins.h und Config.h	Anschlüsse, WLAN und Grenzen
TankDrive / RgbLed / Button	Hardware bedienen und Zustand halten
StatusDisplay	Anzeige auf dem TTGO
TankWebServer	WLAN und HTTP-Routen
WebPage.h und main.cpp	Browseroberfläche und Programmablauf

Klasse, Objekt und Methode

Schnittstelle: RgbLed.h

```
class RgbLed {
public:
  void begin();
  void green() {
    setColor(0, 255, 0);
  }
  // weitere Methoden ...
};
```

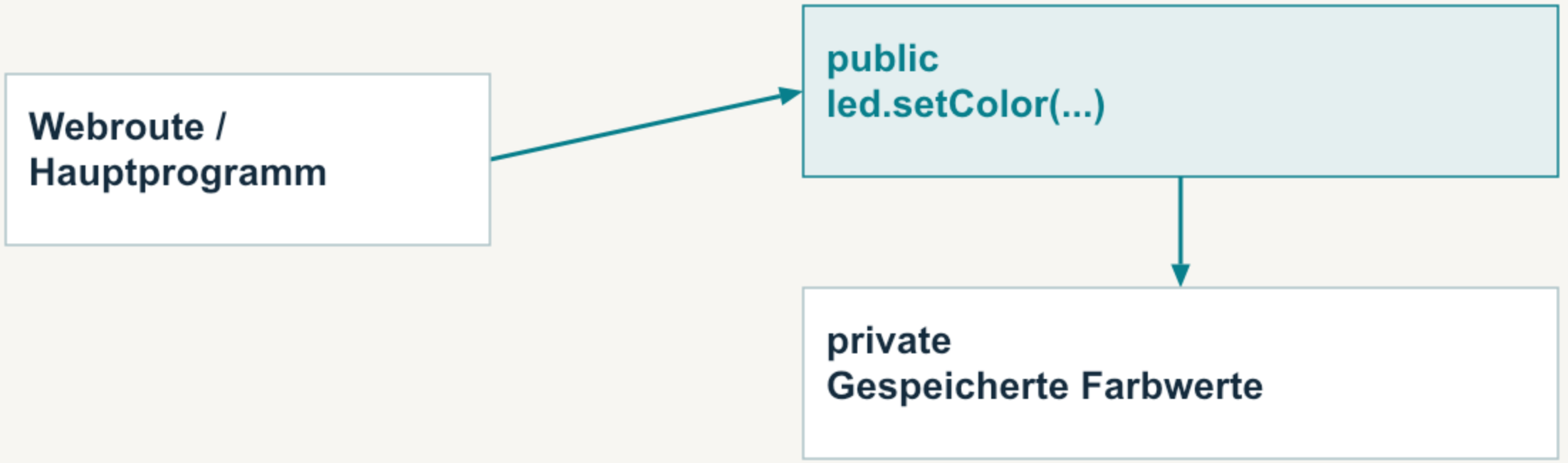
Benutzung: main.cpp

```
RgbLed led;

led.begin();
led.green();
```

Objekt led
Methode green()

public und private begrenzen den Zugriff



Die Klasse hält zusammen, was zur gleichen Hardwareaufgabe gehört.

main.cpp verbindet die Bausteine

```
// setup(): einmal beim Start
drive.begin();
led.begin();
button.begin();
display.begin();
web.begin();
```

```
// loop(): immer wieder
button.update();

if (button.rawPressed())
  drive.arm(false);

web.update();
// Display aktualisieren ...
```

Ausschnitte: Im Original bleiben Initialisierung und Fehlerbehandlung vollständig erhalten.

Live-Demo 2: die Tank-Websteuerung

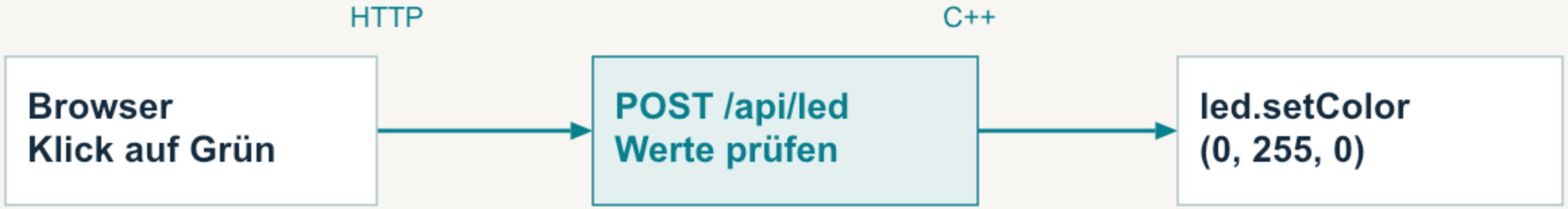
1. RGB-Farbe im Browser ändern
2. Echten Taster drücken, Zähler beobachten
3. Fahren freigeben und Taste gedrückt halten
4. Loslassen und Stopp beobachten

Projekt: WebControl, Umgebung: tank_webcontrol

Die HTTP-Schnittstelle des Tanks

Methode	Route	Aufgabe
GET	/	HTML, CSS und JavaScript
GET	/api/status	Zustand als JSON lesen
POST	/api/led	RGB-Werte setzen
POST	/api/arm	Fahrt freigeben oder sperren
POST	/api/drive	Richtung und PWM vorgeben
POST	/api/stop	Motoren stoppen

Ein Klick auf „Grün“



Anfrage: red=0, green=255, blue=0

Antwort: neuer Zustand als JSON

fetch() ist die Rückverbindung zum ESP32

```
await fetch("/api/led", {
  method: "POST",
  headers: {
    "X-Tank-Control": "1",
    "Content-Type":
      "application/x-www-form-urlencoded"
  },
  body: new URLSearchParams({
    red: 0, green: 255, blue: 0
  })
});
```

Dieser JavaScript-Code läuft im Browser. Der C++-Handler läuft auf dem ESP32.

Statusdaten kommen als JSON zurück

```
{
  "armed": false,
  "left": 0,
  "right": 0,
  "buttonPressed": false,
  "buttonCount": 3,
  "red": 0, "green": 255, "blue": 0
}
```

Browser fragt wiederholt

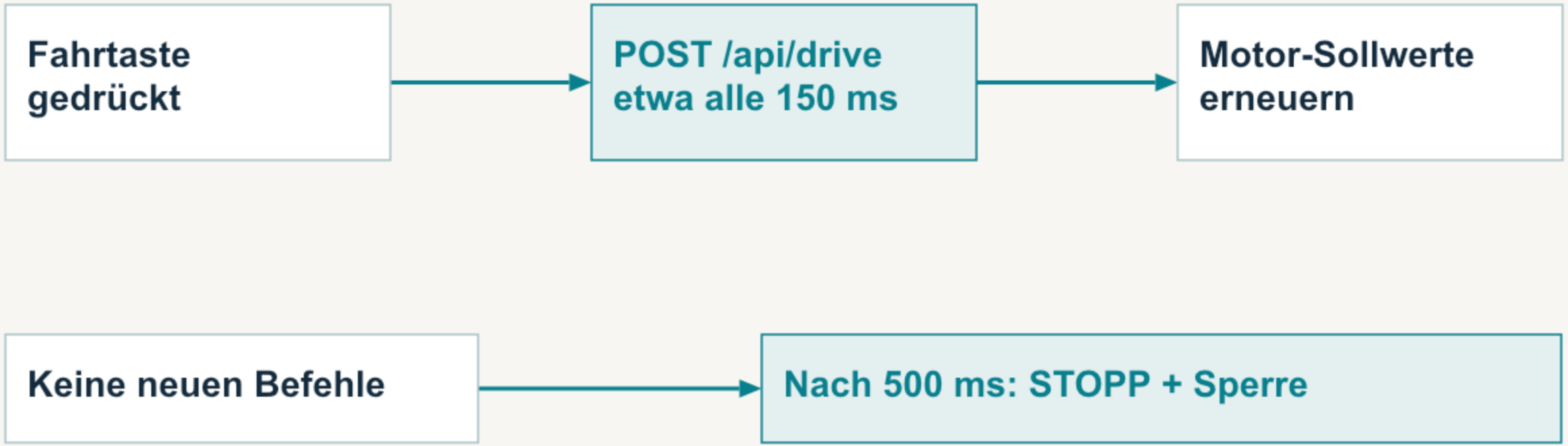
GET /api/status

Antwort auswerten

Anzeige aktualisieren

Nach jeder abgeschlossenen Abfrage wartet der Browser etwa 350 ms.

Fahren braucht laufende Befehle



Der Timer läuft auf dem Gerät und hängt nicht allein von der Webseite ab.

Der echte Taster bleibt lokal wirksam



Der Browser liest denselben Zustand über /api/status.

Lokaler Softwarestopp ersetzt keinen unabhängigen Not-Aus.

KI-Aufgabe: eine eigene Lichtfunktion

Ein zusätzlicher Knopf setzt die LED auf Lila

Eine eigene Route ruft eine Hardwaremethode auf

Die Seite zeigt die aktuelle Farbe aus dem JSON

Erklärfrage: Welche Zeile läuft auf welchem Gerät?

Drei Fragen für das eigene Projekt

Wo läuft der Code?

Welche Nachricht verbindet die Teile?

Was passiert bei einem Verbindungsabbruch?

Code und weiterführende Quellen

Projekt	github.com/gokugiant/tank_chassis
Boards und WLAN	Arduino-Dokumentation und Espressif Wi-Fi API
Web und Server	MDN fetch(), FastAPI und MQTT.org
Lehrdateien	WebControl/LESSON.md und README.md

Anhang: bestätigte Motorbelegung

Bewegung	Enable	HIGH	LOW
Links vorwärts	GPIO 21	GPIO 17	GPIO 22
Links rückwärts	GPIO 21	GPIO 22	GPIO 17
Rechts vorwärts	GPIO 13	GPIO 15	GPIO 2
Rechts rückwärts	GPIO 13	GPIO 2	GPIO 15

Servo deaktiviert. Richtungsnamen gelten aus Sicht des Fahrzeugs.

Anhang: Ablauf der Live-Demos

Schritt	HelloWebServer	Tank-Websteuerung
Arbeitsbereich	01-HelloWebServer	02-TankWebControl
Buildausgabe	hello_webserver	tank_webcontrol
Webseite	Hallo vom Tank!	Fahrt, RGB, Taster
Vorführung	Route /api/team ergänzen	RGB, Taster, dann Fahrt

WLAN: Tank-Lab Passwort: TankLab2026 Adresse: http://192.168.4.1